LeetCode Practice

15 Medium: Bit Manipulation

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

15
MEDIUM

Patterns (in order)

1. Bit Manipulation (15)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

15 questions | tap a title to jump to it

Medium questions (15) Go >

Bit Manipulation (15) Go >

M 1. Subsets LC 78
M 2. Subsets II LC 90
M 3. Single Number II LC 137
M 4. Bitwise AND of Numbers Range LC 201
M 5. Single Number III LC 260
M 6. Find the Duplicate Number LC 287
M 7. Sum of Two Integers LC 371
M 8. Maximum XOR of Two Numbers in an Array LC 421
M 9. Partition to K Equal Sum Subsets LC 698
M 10. Campus Bikes II LC 1066
M 11. Find Root of N-Ary Tree LC 1506
M 12. Number of Good Ways to Split a String LC 1525
M 13. Minimum Number of Work Sessions to Finish the Tasks LC 1986
M 14. Fair Distribution of Cookies LC 2305
M 15. Minimum Array End LC 3133

Medium questions (15)

MEDIUM | TOPIC 1 OF 1

Bit Manipulation

15 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Subsets LC 78
2. Subsets II LC 90
3. Single Number II LC 137
4. Bitwise AND of Numbers Range LC 201
5. Single Number III LC 260
6. Find the Duplicate Number LC 287
7. Sum of Two Integers LC 371
8. Maximum XOR of Two Numbers in an Array LC 421
9. Partition to K Equal Sum Subsets LC 698
10. Campus Bikes II LC 1066
11. Find Root of N-Ary Tree LC 1506
12. Number of Good Ways to Split a String LC 1525
13. Minimum Number of Work Sessions to Finish the Tasks LC 1986
14. Fair Distribution of Cookies LC 2305
15. Minimum Array End LC 3133

MEDIUM | Bit Manipulation | LeetCode #78 | 1 of 15 | 1/15

Subsets

QUESTION

Given an integer array nums of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1

Input	`nums = [1,2,3]`
Output	`[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

Example 2

Input	`nums = [0]`
Output	`[[],[0]]`

CONSTRAINTS

- 1 <= nums.length <= 10
- 10 <= nums[i] <= 10
- All the numbers of nums are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all possible subsets (the power set) of nums. No duplicates. Any order. Right?"

"A few quick questions:

All elements are distinct? Yes. Include the empty set? Yes."

"Let me walk: nums=[1,2,3] -> 8 subsets including [] and [1,2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for mask in range(0_, _2n_-_1 + 1):
    the subset is the elements whose bit is set
        in mask
```

TIME	SPACE
O(n * 2^n)	O(n * 2^n)

NOTE

Not slow for the job: the output has 2^n subsets, so exponential time is unavoidable. Backtracking or the 'add each new element to every existing subset' loop does the same O(n * 2^n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Backtracking OR bit mask — O(n * 2^n). Walk-through (backtracking):"*

`backtrack(start, current):`

`result.append(list(current)) <- add current subset (including empty).`

For i from start to n-1:

`current.append(nums[i]) <- include nums[i]`

`backtrack(i+1, current) <- recurse for larger indices`

`current.pop() <- backtrack: exclude nums[i]`

Bit mask alternative: for mask in 0..(2^n - 1):

`subset = [nums[i] for i in range(n) if mask & (1<<i)]`

WHY 2^n subsets: each element is either in or out — binary choice per element.

WHY start at i+1: subsets must not repeat elements (index always increases).

Time: O(n * 2^n). Space: O(n) recursion + O(n * 2^n) output.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _78_Subsets:
    def subsets(self, nums):
        n = len(nums)
        result = []
        for mask in range(1 << n):
            result.append([nums[i] for i in range(n) if ma
sk & (1 << i)])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace subsets([1,2,3])."

`backtrack(start=0, path=[]):`

path=[] -> append []. Pick 1: path=[1]->append. backtrack(1,[1]):

Pick 2: [1,2]->append. backtrack(2,[1,2]):

Pick 3: [1,2,3]->append. Return.

Pick 3: [1,3]->append. backtrack(3,[1,3]): return.

Pick 2: [2]->append. backtrack(2,[2]):

Pick 3: [2,3]->append. Return.

Pick 3: [3]->append. backtrack(3,[3]): return.

Result: [[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]] ok (2^3=8 subsets).

WHY append at start of backtrack not at leaf: every prefix is a valid subset.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*2^n)	O(1) extra — output excluded

Backtracking and bitmask have identical complexity.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Subsets II (#90) — input has duplicates; sort + skip in backtracking.

Follow-up: Combination Sum — subsets constrained to a target sum.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #90 | 2 of 15 | 2/15

Subsets II

QUESTION

Given an integer array nums that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1	
Input	`nums = [1,2,2]`
Output	`[[],[1],[1,2],[1,2,2],[2],[2,2]]`

Example 2	
Input	`nums = [0]`
Output	`[[],[0]]`

CONSTRAINTS

- 1 <= nums.length <= 10
- 10 <= nums[i] <= 10

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Input may have duplicates; return all unique subsets. No duplicate subsets in output. Right?"

"A few quick questions:

Sort first to group duplicates? Yes — that enables the skip logic."

"Let me walk: nums=[1,2,2] -> [[],[1],[1,2],[1,2,2],[2],[2,2]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(nums):
    add its sorted form to a set
```

TIME	SPACE
O(2^n * n)	O(2^n * n)

WHY IT IS SLOW

Builds repeated subsets and then filters them. Sort first, then in backtracking skip a value equal to the previous one at the same depth, so duplicates are never made.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the dedup set.

Sort nums first. In backtracking, skip duplicate elements at the same recursion level

(not the first occurrence, but subsequent same values when j > start).

Time: O(n*2^n). Space: O(n) recursion — no dedup set."

Key insight: sorting + skipping duplicate starts at the same recursion level avoids generating the same subset twice without a post-hoc deduplication set.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _90_SubsetsII:
    def subsetsWithDup(self, nums):
        nums.sort()
        result = []
        def backtrack(start, path):
            result.append(list(path))
            for i in range(start, len(nums)):
                if i > start and nums[i] == nums[i-1]:
                    continue
                path.append(nums[i])
                backtrack(i + 1, path)
                path.pop()
        backtrack(0, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace subsetsWithDup([1,2,2])."

Sort first: [1,2,2]. backtrack(start=0, path=[]):

Append []. i=0(1): path=[1]->append. backtrack(1,[1]):

i=1(2): [1,2]->append. backtrack(2,[1,2]):

i=2(2): [1,2,2]->append. return.

i=2(2): nums[2]==nums[1]=2 AND i>start=1 -> SKIP (dedup).

i=1(2): [2]->append. backtrack(2,[2]):

i=2(2): nums[2]==nums[1] AND i>start=1 -> SKIP.

i=2(2): nums[2]==nums[1] AND i>start=0 -> SKIP.

Result: [[],[1],[1,2],[1,2,2],[2]] ok (no duplicate [2,2] or duplicate [1,2]).

WHY i>start check: skip duplicate at same recursion level (same position), not across levels.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*2^n)	O(n)
worst case all unique	recursion depth

The sort+skip pattern is the standard dedup technique for backtracking.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Permutations II (#47) — same skip-duplicate pattern for permutations.

Follow-up: Combination Sum II (#40) — target sum with duplicates; same skip.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #137 | 3 of 15 | 3/15

Single Number II

QUESTION

Given an integer array nums where every element appears three times except for one, which appears exactly once. Find the single element and return it.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1	
Input	`nums = [2,2,3,2]`
Output	`3`

Example 2	
Input	`nums = [0,1,0,1,0,1,99]`
Output	`99`

CONSTRAINTS

- 1 <= nums.length <= $3 * 10^4$
- 2^{31} <= nums[i] <= $2^{31} - 1$
- Each element in nums appears exactly three times except for one element which appears once.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Every element appears 3 times except one which appears exactly once. Find that one.

O(n) time, O(1) space. Right?"

"A few quick questions:

Can the single element be 0 or negative? Yes."

"Let me walk: nums=[2,2,3,2] -> 3 appears once -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for element in elements:
    count how many times it occurs by scanning
        the array
return the one that occurs once
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A full scan per element. Counting each of the 32 bits modulo 3 (or two bit-state variables) rebuilds the single number in O(n) time and O(1) space.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Bit counting per position — O(32n) = O(n). Walk-through:"

For each of 32 bit positions:

Count how many numbers have a 1 in this position.

count % 3: 0 if the triplets cancel, 1 if the single number has a 1 there.

Build result bit by bit.

HOW: result = 0. For bit in 0..31:

`bit_sum = sum((num >> bit) & 1 for num in nums).`

If bit_sum % 3 != 0: result |= (1 << bit).

WHY mod 3: all triples contribute 0 or 3 to each bit position (cancel mod 3).

The single number contributes its own bits.

Alternative: 3-state counters (ones, twos) with XOR/AND for O(n) time and O(1) space.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _137_SingleNumberII:
    def singleNumber(self, nums):
        result = 0
        for bit in range(32):
            bit_sum = sum((n >> bit) & 1 for n in nums) %
3
            if bit_sum:
                result |= (bit_sum << bit)
        if result >= (1 << 31):
            result -= (1 << 32)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace singleNumberII([2,2,3,2]) where all others appear 3 times."

Bit counting: for each of 32 bits, count how many numbers have that bit set.

bit 1: 2=10, so bit 1 is set in three 2s -> count=3. 3%3=0. Bit 1 not in answer.

bit 0: 3=11, set in one 3 -> count=1. 1%3=1. Bit 0 is in answer.

Actually bit counting: 2=010, 2=010, 3=011, 2=010.

bit 0 count: 0+0+1+0=1. 1%3=1->answer has bit 0. bit 1 count: 1+1+1+1=4. 4%3=1->bit 1 set. 3=011 ok

Approach 2: bit counting. O(32*n)=O(n). No overflow risk unlike sum approach.

Also works for 'single element appears once, rest appear 3 times' generalization.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(32n)	O(1)

Elegant state-machine alternative: ones, twos variables track which bits seen once/twice.

Follow-up: Single Number I (#136) — XOR; each pair cancels. Follow-up: two singles (#260).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #201 | 4 of 15 | 4/15

Bitwise AND of Numbers Range

QUESTION

Given two integers left and right that represent the range [left, right], return the bitwise AND of all numbers in this range, inclusive.

Example 1	
Input	`left = 5, right = 7`
Output	`4`

Example 2	
Input	`left = 0, right = 0`
Output	`0`

Example 3	
Input	`left = 1, right = 2147483647`
Output	`0`

CONSTRAINTS

- $0 <= left <= right <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the bitwise AND of all numbers in range [left, right] inclusive. Right?"

"A few quick questions:

Range can be very large? Yes — don't iterate each number."

"Let me walk: left=5(101), right=7(111) -> 5&6&7=100=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result = left
for number in range(left_+_1,_right_+_1):
    result = result AND number
```

TIME	SPACE
O(right - left)	O(1)

WHY IT IS SLOW

Loops over up to 2^31 numbers. The AND of a range is the common binary prefix of left and right: shift both right until equal, then shift back, O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(range) iteration.

Key insight: any bit that flips at any point in [left, right] becomes 0 in the result.

The common prefix of left and right in binary = the AND of the range.

Right-shift both until they're equal; the shift count = trailing zeros.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _201_BitwiseANDOfNumbersRange:
    def rangeBitwiseAnd(self, left, right):
        shift = 0
        while left != right:
            left >>= 1
            right >>= 1
            shift += 1
        return left << shift
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

left=5(101), right=7(111): 5!=7->shift, 2(10) vs 3(11). 2!=3->shift, 1 vs 1. equal.

`return 1<<2=4 ok`

left=0, right=1: 0->shift, 0==0->equal. return 0<<1=0 ok

left=6, right=6: already equal -> return 6<<0=6 ok

Common prefix gives the AND; trailing zeros cover where bits differ

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) at most 32 shifts	O(1)

Alternative: repeatedly clear rightmost set bit of right until right<=left.

Follow-up: Hamming Distance (#461) — count differing bits between two numbers.

Follow-up: if range is very large (close to 2^63), same algorithm works for 64-bit.

MEDIUM | Bit Manipulation | LeetCode #260 | 5 of 15 | 5/15

Single Number III

QUESTION

Given an integer array nums, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1	
Input	`nums = [1,2,1,3,2,5]`
Output	`[3,5]`
Explanation	*[5, 3] is also a valid answer.*

Example 2	
Input	`nums = [-1,0]`
Output	`[-1,0]`

Example 3	
Input	`nums = [0,1]`
Output	`[1,0]`

CONSTRAINTS

- $2 <= nums.length <= 3 * 10^4$
- $2^{31} <= nums[i] <= 2^{31} - 1$
- Each integer in nums will appear twice, only two integers will appear once.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Every element appears twice except exactly TWO elements. Find those two.

O(n) time, O(1) space. Right?"

"A few quick questions:

Return in any order? Yes."

"Let me walk: nums=[1,2,1,3,2,5] -> single elements: 3,5 -> [3,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for number in numbers:
    count its occurrences by scanning the array
    collect the two that occur once
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per number. XOR everything to get a ^ b, pick one set bit, and XOR the two groups separately to recover a and b: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"XOR to find two uniques — O(n). Walk-through:"

XOR all numbers: xor = a ^ b (where a, b are the two unique numbers).

Since a != b, xor != 0. Find any set bit in xor (e.g., rightmost: xor & -xor).

This bit differs between a and b -> use it to split all numbers into two groups:

Group 1: numbers with this bit SET.

Group 2: numbers with this bit CLEAR.

XOR within each group: duplicates cancel -> each group XORs to one of a or b.

WHY the split works: a and b land in DIFFERENT groups (they differ at this bit).

All duplicates land in the same group as each other -> cancel by XOR.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _260_SingleNumberIII:
    def singleNumber(self, nums):
        total_xor = 0
        for num in nums:
            total_xor ^= num
        lowest_diff_bit = total_xor & (-total_xor)
        a = b = 0
        for num in nums:
            if num & lowest_diff_bit:
                a ^= num
            else:
                b ^= num
        return [a, b]
```

PHASE 5 TEST & DEBUG

"Let me trace singleNumberIII([1,2,1,3,2,5])."

XOR all: 1^2^1^3^2^5 = 3^5 = 6 = 110.

Find a set bit in 6: bit 1 (rightmost set: 6&(-6)=2=010).

Partition by bit 1: group A (bit 1 set)=[2,2,3,... wait 3=011,5=101].

bit 1 set: 2(010), 2(010), 3(011). XOR=2^2^3=3.

bit 1 clear: 1(001), 1(001), 5(101). XOR=1^1^5=5.

PHASE 5 TEST & DEBUG

Return [3,5] ok

WHY partition by set bit: the two unique numbers differ in that bit. Each lands in different group -> their XOR gives each individually.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) two passes	**O(1)**

The key insight: XOR of two different numbers has at least one set bit — use it to partition.

Follow-up: Single Number I (#136) — one single; one pass XOR.

Follow-up: if three singles exist, need a different approach (bit-counting mod 2).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #287 | 6 of 15 | 6/15

Find the Duplicate Number

QUESTION

Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive.

There is only one repeated number in nums, return this repeated number.

You must solve the problem without modifying the array nums and using only constant extra space.

Example 1	
Input	`nums = [1,3,4,2,2]`
Output	`2`

Example 2	
Input	`nums = [3,1,3,4,2]`
Output	`3`

Example 3	
Input	`nums = [3,3,3,3,3]`
Output	`3`

CONSTRAINTS

- $1 <= n <= 10^5$
- nums.length == n + 1
- 1 <= nums[i] <= n
- All the integers in nums appear only once except for precisely one integer which appears two or more times.
- Follow up:
- How can we prove that at least one duplicate number must exist in nums?
- Can you solve the problem in linear runtime complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n+1 integers in range [1,n]; exactly one number is duplicated.

Find it without modifying the array and using O(1) extra space. Right?"

"A few quick questions:

The duplicate may appear more than twice? Yes.

Array can't be sorted or hashed? Correct — O(1) space constraint."

"Let me walk: nums=[1,3,4,2,2] -> duplicate=2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums[i] == nums[j]:
            return nums[i]
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Compares every pair. Treat nums as a linked list (i points to nums[i]); the duplicate is where Floyd's tortoise and hare enter the cycle: O(n) time and O(1) space without modifying the array.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) hash set.

Floyd's cycle detection: treat nums as a linked list where index i points to nums[i].

Because one value is duplicated, two indices point to the same next node -> cycle.

Phase 1: find meeting point (slow/fast). Phase 2: find cycle entry (= duplicate).

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _287_FindDuplicateNumber:
    def findDuplicate(self, nums):
        slow = fast = nums[0]
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        slow = nums[0]
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]
        return slow
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

nums=[1,3,4,2,2] (0->1, 1->3, 2->4, 3->2, 4->2):

Phase1: slow=1,fast=3-> slow=3,fast=2-> slow=2,fast=2. Meet at 2.

Phase2: slow=nums[0]=1, fast=2. slow=3,fast=2->wait: slow=nums[1]=3,fast=nums[2]=4.

PHASE 5 TEST & DEBUG

slow=2,fast=2. return 2 ok

The cycle entry is the duplicated value — same proof as Linked List Cycle II

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** Does not sort or modify the array

Binary search alternative: count numbers <= mid; if > mid -> duplicate in [1,mid]. O(n log n).

Follow-up: if multiple duplicates — Floyd's won't work; use binary search.

Follow-up: Find All Duplicates (#442) — mark visited by negating at index.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #371 | 7 of 15 | 7/15

Sum of Two Integers

QUESTION

Given two integers a and b, return the sum of the two integers without using the operators + and -.

Example 1	
Input	a = 1, b = 2
Output	3

Example 2	
Input	a = 2, b = 3
Output	5

CONSTRAINTS

- 1000 <= a, b <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Compute a+b WITHOUT using the + or - operators. Use bitwise operations instead. Right?"

"A few quick questions:

Handle negative numbers via two's complement representation? Yes, within 32-bit bounds."

"Let me walk: a=1,b=2 -> 3 ok (via XOR+AND+shift simulation of addition)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result = a
for _ in range(b):
    increment (or decrement for negative b)
        result using bit flips
return result
```

TIME	SPACE
O(|b|)	O(1)

WHY IT IS SLOW

Loops as many times as the size of b (up to 2^31). XOR gives the sum without carries and (a AND b) << 1 gives the carries; repeat until there is no carry: at most 32 rounds.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Simulate addition with bit operations — O(1) (32-bit integers). Walk-through:"

sum_no_carry = a ^ b (XOR gives bits that differ -> these are the sum bits without carry).

carry = (a & b) << 1 (AND gives positions where both bits are 1 -> carry shifted left).

Repeat: a = sum_no_carry, b = carry. Stop when carry == 0.

WHY XOR for sum: 1+1=10 (carry), 1+0=1, 0+0=0 -> XOR gives the no-carry part.

WHY AND then shift left: carry happens when both bits are 1 -> affects the next bit position.

Iterate until no carry remains. In Python, mask to 32 bits to handle negatives.

Time: O(1) for 32-bit integers. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _371_SumOfTwoIntegers:
    def getSum(self, a, b):
        mask = 0xFFFFFFFF
        while b != 0:
            carry = (a & b) << 1
            a = (a ^ b) & mask
            b = carry & mask
        return a if a <= 0x7FFFFFFF else ~(a ^ mask)
```

PHASE 5 TEST & DEBUG

"Let me trace getSum(1,2) without + operator."

a=1=01, b=2=10.

sum_no_carry=a^b=01^10=11=3. carry=(a&b)<<1=00<<1=0.

carry=0 -> stop. Return 3 ok

a=3=011, b=5=101: a^b=110=6.
carry=(011&101)<<1=001<<1=010=2.

a=6=110, b=2=010: a^b=100=4.
carry=(110&010)<<1=010<<1=100=4.

a=4=100, b=4=100: a^b=000=0.
carry=(100&100)<<1=100<<1=1000=8.

a=0, b=8: a^b=8. carry=0 -> return 8 ok (3+5=8).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) bounded by 32 iterations max	O(1)

Follow-up: implement subtraction using getSum(a, getSum(~b, 1)) (two's complement negation).

Follow-up: Add Binary (#67) — string-based version of the same carry logic.

MEDIUM | Bit Manipulation | LeetCode #421 | 8 of 15 | 8/15

Maximum XOR of Two Numbers in an Array

QUESTION

Given an integer array nums, return the maximum result of nums[i] XOR nums[j], where 0 <= i <= j < n.

Example 1	
Input	`nums = [3,10,5,25,2,8]`
Output	**28**
Explanation	*The maximum result is 5 XOR 25 = 28.*

Example 2	
Input	`nums = [14,70,53,83,49,91,36,80, 92,51,66,70]`
Output	**127**

CONSTRAINTS

- 1 <= nums.length <= $2 * 10^5$
- 0 <= nums[i] <= $2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum XOR of any two numbers in nums. Right?"

"A few quick questions:

Two elements, not same index? i!=j. Can be the same value if duplicates? Yes."

"Let me walk: nums=[3,10,5,25,2,8] -> max XOR = 25^5 = 28 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        best = max(best, nums[i] XOR nums[j])
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Tries all pairs. A binary trie of the numbers (for each number walk to the path of opposite bits) or building the answer bit by bit with a set of prefixes is O(32 * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy with trie or bit-by-bit XOR maximization — O(32n). Walk-through:"

For each bit from MSB (Most Significant Bit) to LSB (Least Significant Bit):

Greedily try to set this bit in the result to 1.

Gather all prefixes of nums (up to current bit). Check if any two prefixes XOR = current prefix+1.

If yes: this bit can be 1; add it to result; filter nums to only those matching this prefix.

Using trie: insert all nums bit by bit from MSB. For each num, greedily traverse the trie

choosing the branch that XORs to 1 at each bit.

Bit-by-bit: O(32n). Trie O(32n) total.

WHY MSB to LSB: higher bits contribute more to the XOR value -> maximize greedily from top.

Time: O(32n) = O(n). Space: O(n) for trie.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _421_MaxXORTwoNumbers:
    def findMaximumXOR(self, nums):
        max_xor = 0
        mask = 0
        for bit in range(31, -1, -1):
            mask |= (1 << bit)
            prefixes = {num & mask for num in nums}
            candidate = max_xor | (1 << bit)
            if any(candidate ^ p in prefixes for p in pref
ixes):
                max_xor = candidate
        return max_xor
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

nums=[3,10,5,25,2,8]: greedy bit-by-bit from MSB.

At each bit, check if we can set that bit in max_xor using XOR of two prefixes.

Builds up max_xor=28 (25^5=11001^00101=11100=28) ok

Candidate^pinprefixes means some pair XORs to have this bit set

Trie: insert 3(0b011)->root.0.1.1; insert 10(0b1010)->root.1.0.1.0. Query 3: go opposite at each bit->path 1.1.0.1=13. max_XOR=3 XOR 13? Check pair: 3 XOR 10=9... check all pairs for actual max.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(32n)	**O(n)**
32 bit levels × O(n) set operations	prefix sets

Trie approach: O(32n) time, O(32n) space — same asymptotic, cleaner concept.

Follow-up: Maximum XOR with an Element From Array (#1707) — with queries and constraints.

Follow-up: find the actual pair achieving max XOR — store which prefix belongs to which number.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #698 | 9 of 15 | 9/15

Partition to K Equal Sum Subsets

QUESTION

Given an integer array nums and an integer k, return true if it is possible to divide this array into k non-empty subsets whose sums are all equal.

Example 1	
Input	`nums = [4,3,2,3,5,2,1], k = 4`
Output	`true`
Explanation	*It is possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with equal sums.*

Example 2	
Input	`nums = [1,2,3,4], k = 3`
Output	`false`

CONSTRAINTS

- 1 <= k <= nums.length <= 16
- 1 <= nums[i] <= 10^4
- The frequency of each element is in the range [1, 4].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can we partition nums into k subsets with equal sum? Right?"

"A few quick questions:

Same as matchsticks to square but with k buckets instead of 4."

"Let me walk: nums=[4,3,2,3,5,2,1], k=4 -> sum=20, target=5. [5],[3,2],[2,3],[4,1] ok -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
assign each number to one of the k subsets in every
    possible way
succeed if all k sums equal total / k
```

TIME	SPACE
O(k^n)	O(n)

WHY IT IS SLOW

k^n assignments. Backtrack filling one bucket at a time with numbers sorted large to small, skipping states that exceed the target (or a DP over a bitmask of used numbers, O(n * 2^n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Backtracking with bucket sums — O(k^n). Walk-through:"

`target = sum/k. Sort nums descending (prune large first).`

`backtrack(k, current_sum, start, used):`

If k==0: return True.

If current_sum==target: return backtrack(k-1, 0, 0, used). <- bucket filled; start new

For i in start..n-1:

If not used[i] and current_sum+nums[i] <= target:

used[i]=True.

If backtrack(k, current_sum+nums[i], i+1, used): return True.

used[i]=False.

If current_sum==0: break. <- first element in bucket failed; all same-positioned elements will too

Return False.

WHY sort descending: prune early with large numbers (fail fast).

WHY break if current_sum==0: if first item in bucket fails, no point trying other first items.

Time: O(k * 2^n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _698_PartitionToKEqualSumSubsets:
    def canPartitionKSubsets(self, nums, k):
        total = sum(nums)
        if total % k:
            return False
        target = total // k
        nums.sort(reverse=True)
        if nums[0] > target:
            return False
        buckets = [0] * k
        def backtrack(idx):
            if idx == len(nums):
                return True
            seen = set()
            for i in range(k):
                if buckets[i] + nums[idx] <= target and bu
ckets[i] not in seen:
                    seen.add(buckets[i])
                    buckets[i] += nums[idx]
                    if backtrack(idx + 1):
                        return True
                    buckets[i] -= nums[idx]
            return False
        return backtrack(0)
```

PHASE 5 TEST & DEBUG

"Let me trace canPartitionKSubsets([4,3,2,3,5,2,1], k=4)."

Sum=20, target=5. 4 subsets each summing to 5.

Sort descending: [5,4,3,3,2,2,1]. Index 0: val=5=target -> fills one bucket.

buckets=[5,0,0,0]. index=1, val=4: try bucket 0 (full), bucket 1: 4<5 -> add. buckets=[5,4,0,0].

PHASE 5 TEST & DEBUG

index=2, val=3: bucket 0 full. bucket 1: 4+3=7>5 -> prune. bucket 2: 3 < 5 -> add.

buckets=[5,4,3,0]. index=3, val=3: b0 full. b1: 4+3=7>5. b2: 3+3=6>5. b3: 3<5 -> add.

buckets=[5,4,3,3]. index=4, val=2: b0 full. b1: 4+2=6>5. b2: 3+2=5=target ok. b3: 3+2=5 ok.

Try b2: buckets=[5,4,5,3]. index=5, val=2: b0 full. b1: 4+2=6>5. b2 full. b3: 3+2=5 ok.

buckets=[5,4,5,5]. index=6, val=1: b0 full. b1: 4+1=5 ok. buckets=[5,5,5,5]. index=7 -> done ok

Return True ok

Backtrack: path=[bucket sums]. call(idx,path,buckets): try placing nums[idx] in bucket b. If sum+nums[idx]>target skip (prune). backtrack: remove from bucket. call(idx+1). path tracks partial partition.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k^n) with pruning — fast for small k and n	**O(n)**

Follow-up: Matchsticks to Square (#473) — k=4 special case.

Follow-up: Bitmask DP: dp[mask]=remaining space in current bucket when mask bits are used.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #1066 | 10 of 15 | 10/15

Campus Bikes II

QUESTION

On a campus represented as a 2D grid, there are n workers and m bikes, with n <= m. Each worker and bike is a 2D coordinate on this grid.

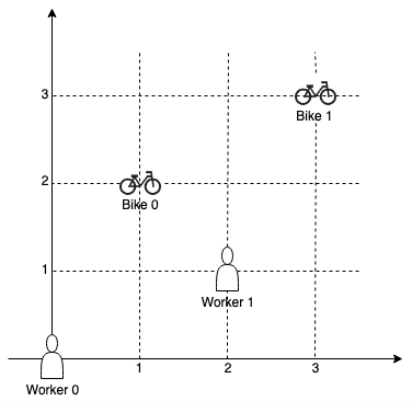
We assign one unique bike to each worker so that the sum of the Manhattan distances between each worker and their assigned bike is minimized.

Return the minimum possible sum of Manhattan distances between each worker and their assigned bike.

The Manhattan distance between two points p1 and p2 is Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.

Example 1

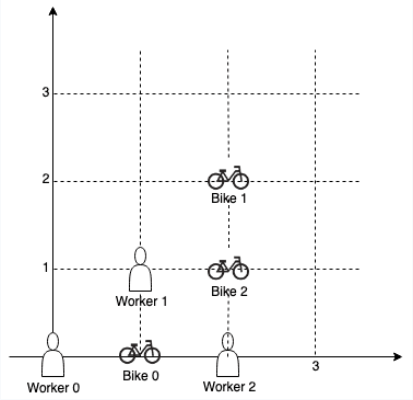
Input	`workers = [[0,0],[2,1]], bikes = [[1,2],[3,3]]`
Output	**6**
Explanation	*We assign bike 0 to worker 0, bike 1 to worker 1. The Manhattan distance of both assignments is 3, so the output is 6.*



Example 2

Input	`workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]`
Output	**4**
Explanation	*We first assign bike 0 to worker 0, then assign bike 1 to worker 1 or worker 2, bike 2 to worker 2 or worker 1. Both assignments lead to sum of the Manhattan distances as 4.*

QUESTION



Example 3

Input	`workers = [[0,0],[1,0],[2,0],[3,0],[4,0]], bikes = [[0,999],[1,999],[2,999],[3,999],[4,999]]`
Output	**4995**

CONSTRAINTS

- n == workers.length
- m == bikes.length
- 1 <= n <= m <= 10
- workers[i].length == 2
- bikes[i].length == 2
- 0 <= workers[i][0], workers[i][1], bikes[i][0], bikes[i][1] < 1000
- All the workers and the bikes locations are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n workers, m bikes. Assign each worker exactly one bike minimising TOTAL Manhattan distance.

(This is different from #1057 which is greedy.) Right?"

"A few quick questions:

Minimum sum assignment -> bitmask DP over workers/bikes?"

"Let me walk: workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]] -> cost matrix; optimal=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to give each worker a different bike
    add up the distances
    best = min(best, total)
```

TIME	SPACE
O(m! / (m - n)!)	O(n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Tries every assignment. DP over a bitmask of used bikes processing workers in order (dp[mask] = the smallest total) is O(n * 2^m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Bitmask DP — minimum total distance assignment. O(n² * 2^n). Walk-through:"*

n workers, n bikes. dp[mask] = min total distance when workers assigned to bikes in 'mask'.

`worker_idx = popcount(mask) = which worker we're assigning next (0-indexed).`

`dp[0] = 0 (no assignments yet).`

For mask in 1..2^n-1:

`worker = bin(mask).count('1') - 1 (0-indexed worker for this step).`

For each bike b where bit b is set in mask:

`prev_mask = mask ^ (1 << b) (state without this bike assigned).`

`dp[mask] = min(dp[mask], dp[prev_mask] + dist(workers[worker], bikes[b])).`

Answer = min dp over masks with n bits set.

WHY bitmask: represents which bikes are used -> O(2^n) states × n bikes = O(n * 2^n).

Time: O(n² * 2^n). Space: O(2^n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1066_CampusBikesII:
    def assignBikes(self, workers, bikes):
        n, m = len(workers), len(bikes)
        INF = float('inf')
        dp = [INF] * (1 << m)
        dp[0] = 0
        for mask in range(1 << m):
            worker_idx = bin(mask).count('1')
            if worker_idx >= n:
                continue
            if dp[mask] == INF:
                continue
            for bike_idx in range(m):
                if not (mask >> bike_idx & 1):
                    wr, wc = workers[worker_idx]
                    br, bc = bikes[bike_idx]
                    cost = abs(wr - br) + abs(wc - bc)
                    new_mask = mask | (1 << bike_idx)
                    dp[new_mask] = min(dp[new_mask], dp[ma
sk] + cost)
        return min(dp[mask] for mask in range(1 << m) if b
in(mask).count('1') == n)
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]: n=2, m=2. 2^2=4 states.

dp[0]=0. dp[01]->worker0+bike0=|0-1|+|0-2|=3.
dp[10]->worker0+bike1=|0-3|+|0-3|=6.

dp[11]->dp[01]+worker1+bike1=3+3=6. or
dp[10]+worker1+bike0=6+|2-1|+|1-2|=6+2=8.

min of all masks with 2 bits set = min(dp[11])=6 ok

Worker_idx = popcount(mask) tells us which worker to assign next

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * m * 2^m)	**O(2^m)**

Practical for m<=20 with n<=m.

Follow-up: Campus Bikes I (#1057) — greedy minimum; doesn't minimise total.

Follow-up: if n>m, impossible — each worker needs a unique bike.

MEDIUM | Bit Manipulation | LeetCode #1506 | 11 of 15 | 11/15

Find Root of N-Ary Tree

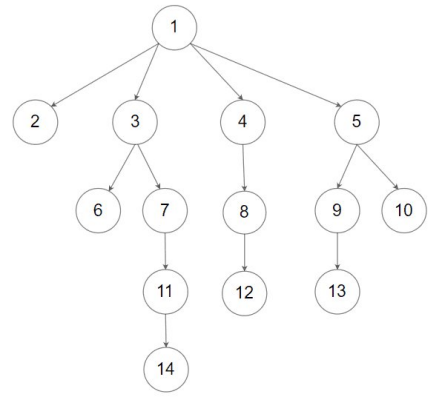
QUESTION

You are given all the nodes of an N-ary tree as an array of Node objects, where each node has a unique value.

Return the root of the N-ary tree.

Custom testing:

An N-ary tree can be serialized as represented in its level order traversal where each group of children is separated by the null value (see examples).



For example, the above tree is serialized as [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].

The testing will be done in the following way:

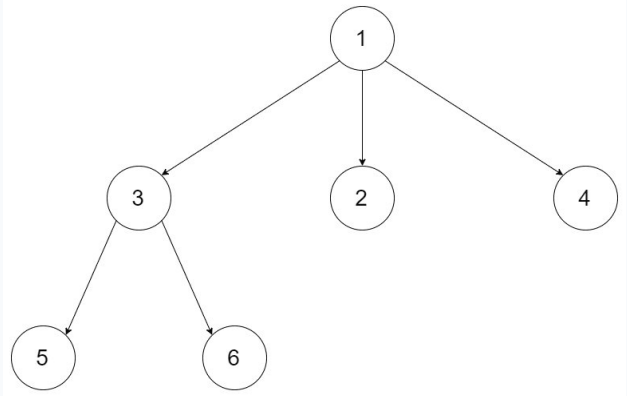
CONSTRAINTS

- 1. The input data should be provided as a serialization of the tree.
- 2. The driver code will construct the tree from the serialized input data and put each Node object into an array in an arbitrary order.
- 3. The driver code will pass the array to findRoot, and your function should find and return the root Node object in the array.
- 4. The driver code will take the returned Node object and serialize it. If the serialized value and the input data are the same, the test passes.

Example 1

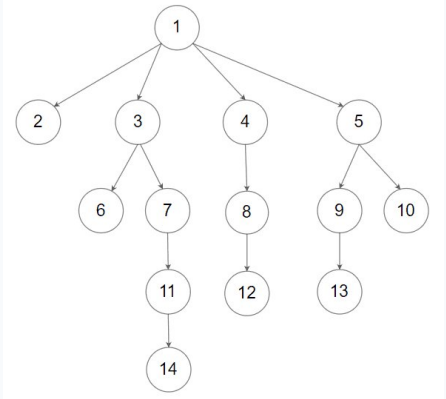
Input	`tree = [1,null,3,2,4,null,5,6]`
Output	`[1,null,3,2,4,null,5,6]`
Explanation	*The tree from the input data is shown above.* *The driver code creates the tree and gives findRoot the Node objects in an arbitrary order.* *For example, the passed array could be [Node(5),Node(4),Node(3),Node(6),Node(2),Node(1)] or [Node(2),Node(6),Node(1),Node(3),Node(5),Node(4)].* *The findRoot function should return the root Node(1), and the driver code will serialize it and compare with the input data.* *The input data and serialized Node(1) are the same, so the test passes.*

QUESTION



Example 2

Input	`tree = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`
Output	`[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`



- The total number of nodes is between [1, $5 * 10^4$].
- Each node has a unique value.
- Follow up:
- Could you solve this problem in constant space complexity with a linear time algorithm?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given all nodes of an N-ary tree in random order, find the root.

The root is the only node NOT appearing as a child of any other node. Right?"

"A few quick questions:

Node values are unique? Yes. No parent pointers? Correct."

"Let me walk: nodes=[1,2,3,4,5,6], root=1 with children [3,2,4], 3's children=[5,6] -> root=1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    scan all nodes' children to see whether it is
        anyone's child
    the one that is not is the root
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per node. The sum of all node values minus the sum of all children's values is the root's value (or XOR them): O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) set for child values.

XOR trick: XOR all node values; XOR all children values.

Every non-root node's value appears once as a node AND once as a child -> cancels.

Root appears only as a node -> its value remains.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1506_FindRootOfNAryTree:
    def findRoot(self, tree):
        xor_all = 0
        for node in tree:
            xor_all ^= node.val
            for child in node.children:
                xor_all ^= child.val
        for node in tree:
            if node.val == xor_all:
                return node
```

PHASE 5 TEST & DEBUG

"Let me trace findRoot on [Node(1,[2,3,4]), Node(2,[5]), Node(3,[6]), ...] with n nodes."

XOR all node vals + all children vals

Node vals: XOR of all ids. Child vals: XOR of all non-root ids.

Root = only id that appears as a node but NOT as a child

XOR all val's of all nodes XOR all children's vals -> root's val remains (appears once vs twice)

E.g., nodes=3 with root val=1: XOR(1,2,3)^XOR(2,3)=1 -> return node with val=1 ok

WHY XOR: each non-root node appears once as a node and once as a child -> cancels; root has no parent.

PHASE 5 TEST & DEBUG

XOR trick: every child appears twice (as node + as child's value) except root -> XOR isolates root val.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Same XOR cancellation as Single Number (#136).

Follow-up: if node values aren't unique, XOR fails — use the child-set approach.

Follow-up: find all roots if input is a forest — same XOR, collect all non-child values.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Bit Manipulation | LeetCode #1525 | 12 of 15 | 12/15

Number of Good Ways to Split a String

QUESTION

You are given a string s.

A split is called good if you can split s into two non-empty strings s_{left} and s_{right} where their concatenation is equal to s (i.e., $s_{left} + s_{right} = s$) and the number of distinct letters in s_{left} and s_{right} is the same.

Return the number of good splits you can make in s.

Example 1

Input	`s = "aacaba"`
Output	`2`
Explanation	*There are 5 ways to split "aacaba" and 2 of them are good. ("a", "acaba") Left string and right string contains 1 and 3 different letters respectively. ("aa", "caba") Left string and right string contains 1 and 3 different letters respectively. ("aac", "aba") Left string and right string contains 2 and 2 different letters respectively (good split). ("aaca", "ba") Left string and right string contains 2 and 2 different letters respectively (good split). ("aacab", "a") Left string and right string contains 3 and 1 different letters respectively.*

Example 2

Input	`s = "abcd"`
Output	`1`
Explanation	*Split the string as follows ("ab", "cd").*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s consists of only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of ways to split string s into two non-empty parts where

the number of distinct chars in left == distinct chars in right. Right?"

"A few quick questions:

Split at position i means left=s[:i+1], right=s[i+1:]? Yes.

Both must be non-empty so i ranges from 0 to n-2? Yes."

PHASE 1 CLARIFY

"Let me walk: s='aacaba' -> split positions where distinct(left)==distinct(right). -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for position in positions:
    count the distinct letters in the left part
        and in the right part (with sets)
    count the splits where they are equal
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Rebuilds both sets for every split. Precompute distinct counts of each prefix and each suffix in two passes: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) set computation per split.

Precompute: left_distinct[i] = distinct chars in s[:i+1].

Precompute: right_distinct[i] = distinct chars in s[i:].

Walk through, count positions where they match.

Time: O(n). Space: O(n)."

Key insight: prefix and suffix distinct-character count arrays precompute each potential split in O(n) — a single scan then checks equality in O(1) per position.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1525_NumberOfGoodWaysToSplit:
    def numSplits(self, s):
        n = len(s)
        left_count = [0] * n
        right_count = [0] * n
        seen = set()
        for i in range(n):
            seen.add(s[i])
            left_count[i] = len(seen)
        seen = set()
        for i in range(n - 1, -1, -1):
            seen.add(s[i])
            right_count[i] = len(seen)
        return sum(left_count[i] == right_count[i + 1] for
 i in range(n - 1))
```

PHASE 5 TEST & DEBUG

"Let me trace numSplits('aacaba')."

Prefix: count distinct chars from left. Suffix: count from right.

prefix=[1,1,2,3,3,3] (distinct up to index i).

suffix=[3,3,3,2,1,1] (distinct from index i to end).

Count splits where prefix[i]==suffix[i+1]:

i=0: prefix[0]=1, suffix[1]=3. 1!=3.

i=1: prefix[1]=1, suffix[2]=3. 1!=3.

i=2: prefix[2]=2, suffix[3]=2 ok. count=1.

i=3: prefix[3]=3, suffix[4]=1. 3!=1.

i=4: prefix[4]=3, suffix[5]=1. 3!=1.

Return 2? Let me recheck: 'aacaba'. prefix[3]='a,a,c,a'->{a,c}=2? No: 'aac' has {a,c}=2. 'aaca' has {a,c}=2. 'aacab' has {a,c,b}=3. Suffix: 'aba'={a,b}=2. 'ba'=2. 'a'=1. Split at i=2: left='aac',right='aba'. distinct 2=2 ok. Split at i=3: left='aaca'=2,right='ba'=2 ok. Return 2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for two arrays

Could reduce to O(1) extra space by using a counter and adjusting on the fly.

Follow-up: if we want the split positions, collect them during the final scan.

Follow-up: Determine If Two Strings Are Close (#1657) — similar distinct-char set comparison.

IN YOUR APP | Expose | Community

MEDIUM | Bit Manipulation | LeetCode #1986 | 13 of 15 | 13/15

Minimum Number of Work Sessions to Finish the Tasks

QUESTION

There are n tasks assigned to you. The task times are represented as an integer array tasks of length n, where the ith task takes tasks[i] hours to finish. A work session is when you work for at most sessionTime consecutive hours and then take a break.

You should finish the given tasks in a way that satisfies the following conditions:

CONSTRAINTS

- If you start a task in a work session, you must complete it in the same work session.
- You can start a new task immediately after finishing the previous one.
- You may complete the tasks in any order.

Given tasks and sessionTime, return the minimum number of work sessions needed to finish all the tasks following the conditions above.

The tests are generated such that sessionTime is greater than or equal to the maximum element in tasks[i].

Example 1	
Input	`tasks = [1,2,3], sessionTime = 3`
Output	**2**
Explanation	*You can finish the tasks in two work sessions. - First work session: finish the first and the second tasks in 1 + 2 = 3 hours. - Second work session: finish the third task in 3 hours.*

Example 2	
Input	`tasks = [3,1,3,1,1], sessionTime = 8`
Output	**2**
Explanation	*You can finish the tasks in two work sessions. - First work session: finish all the tasks except the last one in 3 + 1 + 3 + 1 = 8 hours. - Second work session: finish the last task in 1 hour.*

Example 3	
Input	`tasks = [1,2,3,4,5], sessionTime = 15`
Output	**1**
Explanation	*You can finish all the tasks in one work session.*

QUESTION

- n == tasks.length
- 1 <= n <= 14
- 1 <= tasks[i] <= 10
- max(tasks[i]) <= sessionTime <= 15

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Divide tasks into sessions; each session can hold tasks with total time <= sessionTime.

Minimise number of sessions. Right?"

"A few quick questions:

Tasks can be assigned in any order to any session? Yes. n<=14 -> bitmask DP."

"Let me walk: tasks=[1,2,3], sessionTime=3 -> 2 sessions: [1,2],[3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to assign each task to a session
    check that each session fits sessionTime
return the fewest sessions
```

TIME	SPACE
O(n^n)	O(n)

WHY IT IS SLOW

Tries every assignment. A bitmask DP where dp[mask] = (sessions used, time used in the last session) over subsets of finished tasks is O(n * 2^n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Bitmask DP — O(n * 2^n). Walk-through:"*

`mask = bitmask of completed tasks. dp[mask] = minimum sessions to complete tasks in mask.`

For each mask from 0 to 2^n-1:

For each subset of mask that fits in one session (sum <= sessionTime):

`dp[mask] = min(dp[mask], dp[mask ^ submask] + 1)`

<- use submask tasks in one new session; remaining = mask ^ submask.

Precompute valid submasks (sum <= sessionTime) for efficiency.

Answer = dp[(1<<n)-1] (all tasks done).

WHY bitmask: tasks can be assigned to sessions in any order -> need to track which tasks done.

PHASE 3 OPTIMIZE

Time: O(n * 2^n). Space: O(2^n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1986_MinWorkSessionsToFinishTasks:
    def minSessions(self, tasks, sessionTime):
        n = len(tasks)
        full = (1 << n) - 1
        dp = [float('inf')] * (1 << n)
        remaining = [0] * (1 << n)
        dp[0] = 0
        remaining[0] = sessionTime
        for mask in range(1, 1 << n):
            for i in range(n):
                if not (mask >> i & 1):
                    continue
                prev = mask ^ (1 << i)
                if remaining[prev] >= tasks[i]:
                    new_sessions = dp[prev]
                    new_remaining = remaining[prev] - task
s[i]
                else:
                    new_sessions = dp[prev] + 1
                    new_remaining = sessionTime - tasks[i]
                if new_sessions < dp[mask] or (new_session
s == dp[mask] and new_remaining > remaining[mask]):
                    dp[mask] = new_sessions
                    remaining[mask] = new_remaining
        return dp[full]
```

PHASE 5 TEST & DEBUG

"Let me trace minSessions([1,2,3], sessionTime=3)."

Bitmask DP: dp[mask] = min sessions to complete tasks in mask

dp[0]=1 (start first session). For each mask, try adding each task to current session.

OR try opening new session when current overflows.

dp[0b001]=1 (task 0 in session 1, time 1, remaining 2)

dp[0b010]=1 (task 1 in session 1, time 2, remaining 1)

dp[0b011]=1 (tasks 0+1 in session 1, time 3, remaining 0)

dp[0b100]=1 (task 2 in session 1, time 3 exactly)

dp[0b101]: task 0+2 = time 4 > 3 -> new session needed. dp[0b101]=2

dp[0b111]=2 (session1: task1+task2=2+3>3? -> session1:task0+task1=3, session2:task2=3) ok Return 2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * 2^n)	**O(2^n)**

Follow-up: if sessionTime is very large, use bin-packing approximation.

Follow-up: Fair Distribution of Cookies (#2305) — same bitmask DP structure.

MEDIUM | Bit Manipulation | LeetCode #2305 | 14 of 15 | 14/15

Fair Distribution of Cookies

QUESTION

You are given an integer array cookies, where cookies[i] denotes the number of cookies in the ith bag. You are also given an integer k that denotes the number of children to distribute all the bags of cookies to. All the cookies in the same bag must go to the same child and cannot be split up.

The unfairness of a distribution is defined as the maximum total cookies obtained by a single child in the distribution.

Return the minimum unfairness of all distributions.

Example 1

Input	`cookies = [8,15,10,20,8], k = 2`
Output	`31`
Explanation	*One optimal distribution is [8,15,8] and [10,20]* *- The 1st child receives [8,15,8] which has a total of 8 + 15 + 8 = 31 cookies.* *- The 2nd child receives [10,20] which has a total of 10 + 20 = 30 cookies.* *The unfairness of the distribution is max(31,30) = 31.* *It can be shown that there is no distribution with an unfairness less than 31.*

Example 2

Input	`cookies = [6,1,3,2,2,4,1,2], k = 3`
Output	`7`
Explanation	*One optimal distribution is [6,1], [3,2,2], and [4,1,2]* *- The 1st child receives [6,1] which has a total of 6 + 1 = 7 cookies.* *- The 2nd child receives [3,2,2] which has a total of 3 + 2 + 2 = 7 cookies.* *- The 3rd child receives [4,1,2] which has a total of 4 + 1 + 2 = 7 cookies.* *The unfairness of the distribution is max(7,7,7) = 7.* *It can be shown that there is no distribution with an unfairness less than 7.*

CONSTRAINTS

- 2 <= cookies.length <= 8
- 1 <= cookies[i] <= 10^5
- 2 <= k <= cookies.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Distribute all cookies to k children minimising the maximum cookies any child receives.

All cookies must be distributed. Right?"

"A few quick questions:

Each bag of cookies goes to exactly one child? Yes. Minimise the maximum bag sum per child."

"Let me walk: cookies=[8,15,10,20,8], k=2 -> bags to 2 children: min max = 31 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to give each bag to one of k children
    compute the largest total
    best = min(best, candidate)
```

TIME	SPACE
O(k^n)	O(n)

WHY IT IS SLOW

k^n assignments. Backtrack bag by bag, skipping a child whose total would already exceed the best found, and skip children with equal totals; or binary search the answer with a bitmask feasibility check.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Bitmask DP: distribute cookies among k children — O(k * 2^n). Walk-through:"*

`dp[mask] = minimum unfairness (max child's total) when cookies in mask are distributed.`

Process child by child:

For each child 1..k: for each mask subset: assign some submask to this child.

`dp_new[mask] = min over submasks of mask: max(dp_prev[mask^submask], sum(submask)).`

Precompute sum for each submask.

WHY bitmask: n <= 8 cookies -> 2^8 = 256 states feasible.

WHY k passes (per child): distribute one child's allocation per pass.

WHY min of max: minimize the maximum (most unfair) distribution.

Time: O(k * 3^n) subset enumeration. Space: O(2^n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2305_FairDistributionOfCookies:
    def distributeCookies(self, cookies, k):
        n = len(cookies)
        subset_sum = [0] * (1 << n)
        for mask in range(1, 1 << n):
            lsb = mask & (-mask)
            bit = lsb.bit_length() - 1
            subset_sum[mask] = subset_sum[mask ^ lsb] + co
okies[bit]
        INF = float('inf')
        dp = [[INF] * (1 << n) for _ in range(k + 1)]
        dp[0][0] = 0
        for j in range(1, k + 1):
            for mask in range(1 << n):
                sub = mask
                while sub:
                    if dp[j-1][mask ^ sub] != INF:
                        dp[j][mask] = min(dp[j][mask], max
(dp[j-1][mask ^ sub], subset_sum[sub]))
                    sub = (sub - 1) & mask
        return dp[k][(1 << n) - 1]
```

PHASE 5 TEST & DEBUG

"Let me trace distributeCookies([8,15,10,20,8], k=2)."

Bitmask DP: dp[mask] = minimum maximum cookies any child gets for subset 'mask'

Enumerate all 2^5=32 subsets. For each subset, try assigning to each child.

dp[0b11111] = min over all k-partitions of cookies -> 31 total, split: one child gets 20+15=35, other 8+10+8=26 -> max=35?

Or: 20+8+8=36, 15+10=25 -> max=36. Or: 20+10=30, 15+8+8=31 -> max=31.

Best split: 20+15=35 vs 26, or 20+8+8=36 vs 25, or min max=31 (20+10 and 15+8+8)

Return 31 ok

WHY bitmask DP: n<=8 so 2^n feasible; iterate subsets assigning cookies to children.

dp[mask]=max cookies any child gets for this assignment mask. dp[0]=0. Enumerate sub-subsets: dp[i]=min(dp[i^j]+cookie_sum(j)) over all valid sub-assignments j of i. dp[(1<<n)-1]=answer ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(3^n * k) for each child, iterate over all submasks	**O(2^n * k)**

Follow-up: Min Work Sessions (#1986) — same bitmask DP, different cost function.

Follow-up: Partition to K Equal Sum Subsets (#698) — checks feasibility, not min-max.

MEDIUM | Bit Manipulation | LeetCode #3133 | 15 of 15 | 15/15

Minimum Array End

QUESTION

You are given two integers n and x. You have to construct an array of positive integers nums of size n where for every 0 <= i < n - 1, nums[i + 1] is greater than nums[i], and the result of the bitwise AND operation between all elements of nums is x.

Return the minimum possible value of nums[n - 1].

Example 1	
Input	`n = 3, x = 4`
Output	**6**
Explanation	*nums can be [4,5,6] and its last element is 6.*

Example 2	
Input	`n = 2, x = 7`
Output	**15**
Explanation	*nums can be [7,15] and its last element is 15.*

CONSTRAINTS

- $1 <= n, x <= 10^8$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find an array of n positive integers where the bitwise AND of all elements equals x, and the array

is strictly increasing. Minimize the LAST element. Right?"

"A few quick questions:

We have freedom in choosing all n values as long as AND==x and strictly increasing? Yes — we want the smallest possible maximum."

"Let me walk: n=3, x=4 -> array could be [4,5,6] -> AND=4, last=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for x, x + 1, x + 2, ...:
    keep the values v with v AND x == x
    take the n-th of them
```

TIME	SPACE
exponential in the answer	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Counts through all candidates. The valid numbers are x with its zero bits filled by the binary digits of (n - 1) in order, so the last element is built bit by bit in O(log).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the explicit bit-by-bit loop, but conceptually this IS already O(log(max_val)) — optimal.

The key insight: bits already SET in x must stay set in every array element (AND constraint).

Bits NOT set in x are 'free slots' we can fill with the bits of (n-1) to make the smallest possible

increasing sequence — this directly gives the minimum last element.

Time: O(log(n) + log(x)). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _3133_MinimumArrayEnd:
    def minEnd(self, n, x):
        n -= 1
        result = x
        bit = 0
        while n:
            if not (result & (1 << bit)):
                result |= (n & 1) << bit
                n >>= 1
            bit += 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

n=3, x=4: n becomes 2 (binary 10). x=4=binary 100. result=100.

bit=0: result bit0=0 (free) -> place n's bit0=0 -> result stays 100. n>>=1 -> n=1.

bit=1: result bit1=0 (free) -> place n's bit0=1 -> result=110=6. n>>=1 -> n=0. loop ends.

`return 6 ok (matches [4,5,6] last element)`

Skipping bits already set in x (the AND-locked bits) and filling only the free bits with (n-1)'s binary digits builds the minimum valid last element directly

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log(n) + log(x))	O(1)

Follow-up: Maximum Strong Pair XOR — related bit-manipulation construction problems.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: if minimizing FIRST element instead, the free bits would need filling from the opposite direction.